DEDAN KIMATHI UNIVERSITY OF TECHNOLOGY

SOFTWARE TESTING & QUALITY ASSURANCE REPORT

University Course Registration System (Flask & SQLite Backend)

Course Code:	CCS 4201 - Software Testing & QA
Project Name:	Student Course Registration System (DeKUT CSIT)
Lead QA Engineer:	Patrick Muli (Reg: C026-01-0001/2023)
Testing Window:	August 10 - August 11, 2026
Status:	**FAILED VERIFICATION (Requires Major Fixes)**

1. System Overview

The target of this Quality Assurance assessment is the recently updated **DeKUT Student Course Registration System**. The system has been refactored from a client-side in-memory mockup to a persistent 3-tier application utilizing a Python Flask API service communicating with an SQLite relational database file (registration.db). The application provides student login, signup account creation, an active credit tracker, a course catalog, schedule clash checking, prerequisite checking, and a dynamic graphical weekly timetable grid.

2. Testing Objectives & Scope

The QA assessment was executed to evaluate the compliance, reliability, and security of the system. Testing was scoped strictly to verify functional workflows (login, registration, dropping), validation algorithms (timetable clash detection, seat limits, credit caps), database integrity (foreign keys, unique index constraints), and basic API security vulnerabilities (unauthenticated endpoint access, plaintext database storage).

3. Testing Methodology & Environment

The testing strategy combined Black-box functional validation on the browser interface with Grey-box API interrogation on the port 5000 REST services, and database validation using DB Browser for SQLite on the binary registration.db file. Verification scripts were utilized to monitor constraint behavior in real-time.

OS Platform	Windows Server 2025 / Windows 11 Enterprise
Web Server Interface	Python http.server (Port 8080)
Database Engine	SQLite v3 (Local File: registration.db)
Application Server	Python Flask v3.0 (Port 5000)
Web Browser Environment	Google Chrome v127 (Standard client Engine)
Test Management Tools	Trello project board, dump_db.py SQLite verification utility

4. Test Execution Summary

Below is a breakdown of the 30 test cases executed against the system:

Status	Count	Percentage
Passed	26	86.67%
Failed	4	13.33%
Blocked	0	0.00%
Not Executed	0	0.00%
Total Cases	30	100.00%

5. Detailed Bug Reports

The following security vulnerabilities and functional defects were verified during analysis:

BUG-01: Plaintext Password Storage in SQLite Users Table	
Module:	Security / Authentication
Steps to Reproduce:	1. Register a student via signup page. 2. Open SQLite database using DB Browser for SQLite. 3. Browse data in table 'users'. 4. View password column.
Expected:	Passwords must be securely hashed using bcrypt/scrypt algorithms before storage.
Actual:	Passwords are saved as raw, readable plain text (e.g. '123456' visible in database).
Severity / Priority:	Critical / P1
Evidence Location:	File: app.py (line 103: data['password'] inserted raw into database)

BUG-02: Lack of Authentication Token / Session Validation on API Endpoints	
Module:	Security / API
Steps to Reproduce:	1. Launch Flask app. 2. Run API GET query directly to '/api/courses?user_id=1' or POST to '/api/toggle-course' using Postman without logging in.
Expected:	Server checks authentication cookie or JWT token and rejects request with 401 Unauthorized.
Actual:	Server processes request and returns/modifies data without any credentials or token validation.

Severity / Priority:	Critical / P1
Evidence Location:	File: app.py (No session check, token checks, or auth decorators exist on endpoints)

BUG-03: Foreign Key Constraint Enforcement Disabled in SQLite Database

Module:	Database Operations
Steps to Reproduce:	1. Enrol user 1 in course 2. 2. Delete user 1 from users table. 3. Query enrolments table for user_id = 1.
Expected:	Cascading deletes trigger automatically, deleting the enrolment mapping.
Actual:	The enrolment mapping remains in the database, resulting in orphaned records and database integrity issues.
Severity / Priority:	High / P2
Evidence Location:	File: app.py (sqlite3 connection does not execute 'PRAGMA foreign_keys = ON')

BUG-04: Missing Server-Side Input Validation on Signup API

Module:	Authentication / Forms
Steps to Reproduce:	1. Send a POST request to '/api/signup' with an empty JSON body or missing required field keys (e.g., firstName).
Expected:	Backend validates input, returns 400 Bad Request with descriptive validation details.
Actual:	Backend throws a python KeyError and returns a generic 500 Internal Server Error page.
Severity / Priority:	High / P2
Evidence Location:	File: app.py (line 103: direct data access without checking key existence or length)

BUG-05: Simulated Prerequisite Checking Bypass for CS Courses

Module:	Course Registration Logic
Steps to Reproduce:	1. Create student account. 2. Attempt to register for 'CCS 3101' (requires 'CCS 2101') in the catalog without having registered for or completed 'CCS 2101'.
Expected:	System identifies missing prerequisite on student record and disables registration.
Actual:	Enrolment is allowed. Backend sets prereqMet to True by default for all courses except CIT 3102.
Severity / Priority:	High / P2
Evidence Location:	File: app.py (lines 159-161: simulated check hardcoded to CIT 3102 bypasses other prerequisites)

BUG-06: Missing NOT NULL Constraints on Enrolment Mapping Columns	
Module:	Database Structure
Steps to Reproduce:	1. Send toggle course POST request with user_id set to null. 2. Verify enrolments table records.
Expected:	Database prevents inserting null user association.
Actual:	Database successfully registers the enrolment, linking a null user to a course.
Severity / Priority:	Medium / P3
Evidence Location:	File: app.py (lines 57-65: enrolments table definition lacks NOT NULL constraints on user_id and course_id)

6. Database & Referential Integrity Verification

Verification queries executed against registration.db proved that SQL level constraints (e.g., UNIQUE email and registration number checks) successfully block double-entries. However, because the Python connection does not enable SQLite foreign key constraints explicitly via PRAGMA foreign_keys = ON;, cascading deletes fail. Deleting a user from the users table leaves orphaned rows in the enrolments table, violating strict referential integrity rules.

7. Security Assessment

The application exhibits critical security vulnerabilities that prevent it from being production-ready:

- **Plaintext Password Storage:** Passwords are saved with zero cryptographic hashing. Anyone with physical access to the DB file can read passwords.
- **Lack of API Authentication:** Endpoints do not implement session checks, tokens (JWT), or CSRF protection. An attacker can perform registrations or view schedules of other users by predicting student IDs.
- **No Backend Input Sanitization:** Raw strings are committed directly to SQLite, making the system prone to persistent script injections (stored XSS).

8. Recommendations

To address the identified quality and security gaps, the following remedies are recommended:

Issue	Recommended Fix	Priority	Module
Plaintext Password Storage inside database	Implement bcrypt / Argon2 password hashing library in Python signup/login APIs	Critical	Security / Authentication
Lack of Authentication Token / Session verification on API endpoints	Implement JWT (JSON Web Tokens) or Flask Session token checks on GET /api/courses and POST /api/toggle-co	Critical	Security / API
SQLite Foreign Key constraints are disabled, causing orphaned enrolment mappings	Execute PRAGMA foreign_keys = ON; in the connection script inside python database connection getter.	High	Database Operations
Missing Server-Side Input Validation in Signup API (causes 500 server errors on empty inputs)	Integrate pydantic / dataclass check key existence and minimum character limits in request JSON body.	High	Authentication / Forms
Mocked prerequisite checking logic bypasses requirements for CS modules	Implement a query looking up completed/registered course history against course prerequisite code inside get_co	High	Course Registration Logic
Missing database level NOT NULL constraints on user_id and course_id in enrolments table	Alter enrolments schema definition to set user_id 'INTEGER NOT NULL' and course_id 'INTEGER NOT NULL'.	[illegible]	Database Structure

9. Final QA Assessment

Overall Status: Requires Major Fixes (NOT READY FOR DEPLOYMENT)

Major Strengths:

• High-fidelity, user-friendly frontend dashboard and dynamic weekly timetable grid.

• Functional validations like credits checking, time clash detection, and course capacity are accurately handled inside Python API services before SQL commits.

Major Weaknesses:

• Plaintext password storage leaves user credentials entirely exposed.

• Total lack of server-side session authentication allows unauthorized course modifications across profiles.

• SQLite foreign keys disabled by default, causing orphaned data entries.

Conclusion:

The system is functional from a user perspective but fails critical security, authorization, and data integrity standards. It must not be deployed to staging/production until plaintext storage is replaced with secure hashing and API authorization guards are implemented.